

---

# Technologies utilisées

---

**Réalisé par :** BITAR Aicha

Année universitaire 2025-2026

# 1 Docker et Docker Compose

## 1.1 Présentation de la technologie

### 1.1.1 Le problème de la portabilité logicielle

Le développement d'une application moderne implique de multiples dépendances : un langage dans une version précise, des bibliothèques, un serveur de base de données, des variables d'environnement, des configurations système. Une application qui fonctionne sur le poste d'un développeur peut échouer sur un autre poste ou sur un serveur de production, en raison de différences dans ces dépendances. Ce problème, résumé par la formule « *cela fonctionne sur ma machine* », a longtemps constitué une source majeure de difficultés dans le déploiement logiciel.

### 1.1.2 La conteneurisation

Docker répond à ce problème par la **conteneurisation**. Un conteneur est une unité logicielle qui empaquette une application avec l'intégralité de ses dépendances (bibliothèques, fichiers système, configurations), de manière à ce qu'elle s'exécute de façon identique dans n'importe quel environnement disposant de Docker.

Le conteneur se distingue de la machine virtuelle par son architecture. Une machine virtuelle embarque un système d'exploitation invité complet, exécuté au-dessus d'un hyperviseur, ce qui la rend lourde et lente à démarrer. Un conteneur, à l'inverse, partage le noyau du système d'exploitation hôte et n'isole que l'espace utilisateur de l'application. Il en résulte des conteneurs légers, démarrant en quelques secondes et consommant peu de ressources.

### 1.1.3 Images et conteneurs

Docker repose sur deux notions fondamentales :

- L'**image** est un modèle en lecture seule qui définit le contenu d'un conteneur : le système de base, les dépendances installées, le code de l'application. Une image est construite à partir d'un fichier de recette appelé **Dockerfile**, qui décrit étape par étape la façon de l'assembler.
- Le **conteneur** est une instance en cours d'exécution d'une image. À partir d'une même image, il est possible de lancer plusieurs conteneurs identiques et indépendants.

La construction d'une image suit un principe de **couches** : chaque instruction du **Dockerfile** génère une couche, et ces couches sont mises en cache. Lorsqu'une image est reconstruite, seules les couches modifiées sont recalculées, ce qui accélère considérablement les reconstructions successives.

#### 1.1.4 Docker Compose

Une application réelle est rarement composée d'un seul conteneur. Elle associe généralement plusieurs services : une base de données, un serveur applicatif, un service de traitement, etc. Gérer manuellement chacun de ces conteneurs, leurs connexions réseau et leur ordre de démarrage deviendrait vite complexe.

**Docker Compose** répond à ce besoin d'orchestration à l'échelle locale. Il permet de décrire, dans un unique fichier déclaratif au format YAML (`docker-compose.yml`), l'ensemble des services d'une application, leurs images, leurs variables d'environnement, leurs volumes de données, leurs ports et leurs dépendances mutuelles. Une seule commande suffit alors à démarrer ou arrêter l'ensemble de l'infrastructure de manière cohérente.

Docker Compose introduit deux mécanismes particulièrement utiles :

- Les **volumes**, qui permettent de faire persister des données au-delà de la durée de vie d'un conteneur, ou de partager des fichiers entre l'hôte et le conteneur.
- Les **profils**, qui permettent de regrouper les services et de n'activer que ceux nécessaires à une tâche donnée, plutôt que de démarrer systématiquement l'ensemble.

## 1.2 Utilisation dans le projet

Dans ce projet, Docker et Docker Compose assurent la reproductibilité complète de l'environnement. L'ensemble de la plateforme (base de données, scripts de traitement, transformations, interface web) s'exécute dans des conteneurs, sans qu'aucune dépendance ne doive être installée manuellement sur le poste de l'utilisateur. Seul Docker est requis.

Le fichier `docker-compose.yml` définit cinq services, chacun associé à un profil correspondant à une phase du pipeline :

- le service **postgres**, qui héberge la base de données relationnelle. Il est le seul service permanent ; il utilise un volume pour faire persister les données et initialise automatiquement le schéma de la base à son premier démarrage ;
- le service **app** (profil **load**), qui exécute les scripts Python de génération et de chargement des données ;
- le service **profiling** (profil **profiling**), qui exécute le script de profilage ;
- le service **dbt** (profil **dbt**), qui exécute les transformations et les tests de qualité ;
- le service **api** (profil **api**), qui expose l'interface web sur le port 8000.

Tous les services applicatifs partagent une même image, construite à partir d'un unique **Dockerfile**, ce qui garantit la cohérence des dépendances Python à travers toutes les phases du projet. Le mécanisme de dépendances de Docker Compose garantit par ailleurs que chaque service ne démarre qu'une fois la base de données prête à recevoir des connexions, grâce à une sonde de santé (*healthcheck*) définie sur le service **postgres**.

L'usage des profils permet d'exécuter chaque phase du pipeline de manière isolée et séquentielle, reflétant fidèlement l'enchaînement logique du traitement : génération, chargement, profilage, transformation, puis exposition via l'interface.

## 2 PostgreSQL

### 2.1 Présentation de la technologie

#### 2.1.1 Les bases de données relationnelles

Une base de données relationnelle organise l'information sous forme de **tables**, composées de lignes et de colonnes. Chaque table représente une entité (un client, un compte, une transaction), chaque ligne un enregistrement, et chaque colonne un attribut. Les tables sont reliées entre elles par des **clés** : une clé primaire identifie de manière unique chaque ligne d'une table, et une clé étrangère établit un lien vers la clé primaire d'une autre table.

Le langage utilisé pour interroger et manipuler ces données est le **SQL** (*Structured Query Language*), un langage déclaratif dans lequel l'utilisateur décrit le résultat souhaité sans préciser la manière de l'obtenir. C'est le moteur de la base qui détermine la stratégie d'exécution optimale.

#### 2.1.2 PostgreSQL, un SGBD relationnel-objet

PostgreSQL est un système de gestion de base de données (SGBD) relationnel-objet, libre et open source, reconnu pour sa robustesse, sa conformité aux standards SQL et sa richesse fonctionnelle. Il est utilisé aussi bien dans des contextes transactionnels que décisionnels.

#### 2.1.3 La notion de schéma

Un **schéma** est un espace de noms à l'intérieur d'une base de données. Il regroupe des tables, des vues et d'autres objets sous un préfixe commun. Deux tables portant le même nom peuvent coexister dans une même base si elles appartiennent à des schémas différents. Cette organisation est particulièrement utile pour séparer logiquement les différentes couches d'un système de données : données brutes, données transformées, données décisionnelles.

#### 2.1.4 Tables et vues

PostgreSQL distingue deux façons de matérialiser un ensemble de données :

- une **table** stocke physiquement les données sur le disque ;
- une **vue** (*view*) ne stocke aucune donnée : elle est définie par une requête SQL, et son contenu est recalculé à chaque interrogation. Une vue agit comme une table virtuelle, offrant une représentation transformée des données sous-jacentes sans les dupliquer.

Cette distinction est essentielle dans la conception d'un pipeline de données : les vues permettent d'exprimer des transformations légères sans coût de stockage, tandis que les tables sont préférables lorsque le résultat doit être persistant ou fréquemment consulté.

### 2.1.5 Contraintes d'intégrité

PostgreSQL permet de définir des **contraintes** qui garantissent la validité des données à l'insertion : la contrainte de clé primaire assure l'unicité, la contrainte NOT NULL interdit les valeurs manquantes, la contrainte UNIQUE empêche les doublons, et la contrainte de **clé étrangère** garantit l'intégrité référentielle en interdisant qu'une ligne référence une entité inexistante.

Ces contraintes sont fondamentales dans un système transactionnel, où elles bloquent les incohérences à la source. Leur activation ou leur désactivation constitue toutefois un choix de conception qui dépend de l'objectif du système, comme l'illustre le présent projet.

## 2.2 Utilisation dans le projet

PostgreSQL constitue le socle de stockage de l'ensemble de la plateforme. Une seule instance héberge la totalité des données, organisée en cinq schémas correspondant aux couches successives du pipeline :

- le schéma **public** héberge la **base de données relationnelle** brute : les sept tables métier (agences, clients, comptes, cartes, GAB, transactions, usages digitaux) ainsi que le journal des décisions ;
- le schéma de **staging** contient les vues de nettoyage ;
- le schéma des **marts de détection** contient les tables enrichies d'un score de qualité ;
- les deux schémas de l'**entrepôt décisionnel** (**dwh\_qualite** et **dwh\_metier**) contiennent les tables finales destinées respectivement au pilotage de la qualité et à l'exploitation métier.

Un choix de conception central du projet concerne les contraintes d'intégrité. Contrairement à un système de production classique, la base relationnelle est **volontairement construite sans contraintes de clés étrangères**. Ce choix est délibéré : l'objectif n'est pas d'empêcher les anomalies d'entrer dans la base, mais de les y laisser pénétrer afin de pouvoir les détecter, les qualifier et les

soumettre à une décision. Si les clés étrangères étaient activées, la base rejetterait silencieusement les enregistrements incohérents lors du chargement, et ces anomalies disparaîtraient sans jamais être analysées ni tracées. Les contraintes d'unicité sur les identifiants (clé primaire) et sur le numéro de CIN sont en revanche conservées, car elles garantissent l'unicité indispensable à l'identification des enregistrements.

Le type JSONB est par ailleurs utilisé dans le journal des décisions pour conserver, sous forme semi-structurée, les données des enregistrements rejetés. Enfin, la distinction entre tables et vues structure l'ensemble du pipeline : les transformations de nettoyage sont matérialisées sous forme de vues (sans coût de stockage), tandis que les tables de détection et les tables de l'entrepôt décisionnel sont matérialisées physiquement, car elles constituent des résultats persistants destinés à être consultés de façon répétée.

## 3 Python et son écosystème

### 3.1 Présentation de la technologie

#### 3.1.1 Le langage Python

Python est un langage de programmation interprété, de haut niveau et à typage dynamique, réputé pour la lisibilité de sa syntaxe. Créé au début des années 1990, il s'est imposé comme l'un des langages de référence dans les domaines de la science des données, du traitement de données et de l'intelligence artificielle.

Cette prééminence ne tient pas au langage seul, mais à son vaste **écosystème de bibliothèques** spécialisées. Là où d'autres langages imposeraient d'écrire manuellement des traitements complexes, Python offre des bibliothèques matures qui encapsulent ces traitements dans des interfaces simples. Le présent projet mobilise plusieurs de ces bibliothèques, présentées ci-dessous.

#### 3.1.2 NumPy

NumPy (*Numerical Python*) est la bibliothèque fondamentale du calcul numérique en Python. Elle introduit un type de données central : le **tableau multidimensionnel** (*array*), qui stocke des nombres de manière contiguë en mémoire.

Cette organisation présente un avantage décisif en termes de performance. Les opérations sur les tableaux NumPy sont **vectorisées** : au lieu de parcourir les éléments un à un par une boucle, elles s'appliquent à l'ensemble du tableau en une seule instruction, exécutée par du code compilé en langage bas niveau. Un calcul sur des millions de valeurs devient ainsi considérablement plus rapide qu'avec des boucles Python classiques. NumPy fournit également des générateurs de nombres aléatoires performants, utiles pour la simulation de données.

### 3.1.3 Pandas

Pandas est la bibliothèque de référence pour la manipulation de données tabulaires. Elle repose sur NumPy et introduit une structure centrale : le **DataFrame**, une table en mémoire composée de lignes et de colonnes, comparable à une feuille de calcul ou à une table de base de données.

Le DataFrame offre un ensemble riche d'opérations : lecture et écriture de fichiers (notamment le format CSV), filtrage, agrégation, jointure entre tables, détection de valeurs manquantes, transformation de colonnes. Ces opérations permettent d'explorer et de préparer des jeux de données de grande taille avec un code concis. Pandas constitue de fait l'outil privilégié pour le profilage et l'analyse exploratoire des données.

### 3.1.4 Faker

Faker est une bibliothèque dédiée à la **génération de données synthétiques**. Elle produit des valeurs fictives mais réalistes : noms, prénoms, adresses, adresses électroniques, numéros de téléphone, dates. Elle prend en charge la localisation : il est possible de générer des données conformes à un contexte géographique et linguistique donné.

Faker est particulièrement utile dans les situations où l'on ne dispose pas de données réelles, que ce soit pour des raisons de confidentialité ou parce que le système est en cours de développement. Elle permet de constituer des jeux de test crédibles, sur lesquels valider le comportement d'une application.

### 3.1.5 SQLAlchemy

SQLAlchemy est une bibliothèque qui fait le lien entre le code Python et une base de données relationnelle. Elle fournit deux niveaux d'utilisation : un niveau d'abstraction élevé (mapping objet-relationnel) et un niveau plus proche du SQL, qui permet d'exécuter des requêtes tout en gérant proprement la connexion à la base.

Un mécanisme important de SQLAlchemy est celui des **requêtes paramétrées**. Plutôt que d'insérer directement des valeurs dans une chaîne SQL — pratique dangereuse qui expose aux injections SQL — SQLAlchemy permet d'utiliser des paramètres nommés, dont les valeurs sont transmises séparément et échappées automatiquement. Cela garantit à la fois la sécurité et la robustesse des requêtes.

## 3.2 Utilisation dans le projet

Python assure l'ensemble des traitements procéduraux du pipeline, en amont des transformations déclaratives. Il intervient à trois étapes clés.

La **génération de données synthétiques** repose sur Faker et NumPy. Faker produit les attributs réalistes des entités bancaires (noms, adresses, contacts des clients), tandis que NumPy pilote la génération des volumes et l'injection

contrôlée des anomalies. Le script de génération ne se contente pas de produire des données valides : il introduit volontairement, dans des proportions maîtrisées, les anomalies que la plateforme devra ensuite détecter (transactions sur comptes inexistantes, cartes actives sur comptes clôturés, clients sans contact valide, etc.). Les données produites sont écrites au format CSV à l'aide de Pandas.

Le **chargement des données** dans PostgreSQL est assuré par Pandas et SQLAlchemy. Pandas lit les fichiers CSV et les charge dans la base, tandis que SQLAlchemy gère la connexion. Une attention particulière est portée au typage des colonnes lors du chargement, afin de préserver l'intégrité des données sensibles comme les numéros de téléphone, qui doivent conserver leur format d'origine.

Le **profilage des données** exploite Pandas pour calculer les métriques de qualité génériques (taux de complétude, cardinalité, distributions, fraîcheur) sur chaque table, et SQLAlchemy pour exécuter les contrôles métier cross-tables directement en SQL. Les requêtes utilisées à cette étape sont systématiquement paramétrées, garantissant leur sécurité.

Enfin, Python assure la **labélisation automatique** des anomalies. Le script correspondant parcourt les règles métier, applique les décisions convenues et gère la propagation en cascade des rejets, en s'appuyant sur SQLAlchemy pour toutes les interactions avec la base. Le caractère procédural de Python se révèle ici indispensable : la logique de cascade, récursive et conditionnelle, ne pourrait être exprimée par une simple requête déclarative.

## 4 dbt (data build tool)

### 4.1 Présentation de la technologie

dbt (*data build tool*) est un outil qui permet d'organiser et de fiabiliser des traitements de données écrits en SQL. Son principe est simple : dbt ne possède pas de moteur de calcul propre et ne déplace jamais les données. Il génère du SQL et le fait exécuter directement par la base de données, en se chargeant de structurer ces traitements, d'en gérer l'enchaînement et d'y appliquer des contrôles automatiques.

Il faut d'emblée préciser le sens du mot « traitement » dans ce contexte, car il ne s'agit jamais de modifier le contenu des données. Avec dbt, on ne réécrit pas une valeur, on ne corrige pas un montant, on n'altère pas une donnée existante. On produit uniquement de **nouvelles représentations** à partir des données brutes : convertir un type, normaliser une casse, calculer une colonne dérivée comme un score ou un indicateur, joindre plusieurs tables, filtrer ou réorganiser un ensemble de lignes. La valeur d'origine reste intacte ; ce que l'on ajoute, ce sont des informations calculées et des mises en forme. Cette nuance est essentielle dans le cadre de ce projet, où les données de reporting ne peuvent en aucun cas être altérées : dbt y sert à **détecter et qualifier** les anomalies, non à les corriger.

#### 4.1.1 Les modèles

L'unité de base de dbt est le **modèle** : un fichier contenant une simple requête **SELECT**. Le nom du fichier devient le nom de l'objet créé dans la base, et dbt se charge d'envelopper automatiquement la requête dans les instructions nécessaires à sa création.

Ainsi, un modèle écrit sous la forme :

```
select client_id, nom, email
from clients
where email is not null
```

est traduit par dbt en :

```
CREATE TABLE mon_schema.mon_modele AS
select client_id, nom, email
from clients
where email is not null;
```

Le développeur se concentre donc sur la logique (le **SELECT**), tandis que dbt gère la création de l'objet. La définition de la structure et son peuplement, traditionnellement séparés, sont ici réunis dans un seul fichier.

#### 4.1.2 La matérialisation : vue ou table

Pour chaque modèle, dbt permet de choisir la façon dont il est concrétisé dans la base :

- en **vue** (**view**) : le modèle devient une vue SQL, sans stockage physique, dont le résultat est recalculé à chaque interrogation ; ce choix convient aux traitements légers et intermédiaires ;
- en **table** (**table**) : le modèle devient une table physique, calculée une fois et stockée sur disque ; ce choix convient aux résultats persistants et fréquemment consultés.

Ce réglage permet d'arbitrer, à chaque niveau, entre le coût de stockage et le coût de recalcul.

#### 4.1.3 Sources, références et ordre d'exécution

dbt gère automatiquement les dépendances entre les traitements, grâce à deux fonctions.

La fonction **source()** désigne une table brute déjà présente dans la base, extérieure à dbt. La fonction **ref()** désigne un autre modèle dbt : lorsqu'un modèle A fait appel à **ref('B')**, dbt comprend que A dépend de B et que B doit donc être construit en premier.

À partir de l'ensemble de ces liens, dbt déduit seul l'**ordre d'exécution** correct de tous les modèles. Le développeur n'a jamais à préciser cet ordre :

il suffit d'exprimer les dépendances, et dbt construit chaque couche au bon moment, à partir d'une seule commande. Cette résolution automatique est un atout déterminant pour la maintenance et l'automatisation du pipeline.

#### 4.1.4 Les expressions dynamiques

Les fichiers dbt ne contiennent pas uniquement du SQL : ils peuvent inclure des expressions dynamiques, reconnaissables à leurs doubles accolades `{{ ... }}`, qui rendent possibles les fonctions `source()` et `ref()` ainsi que diverses configurations. Avant l'exécution, dbt **compile** ces fichiers : il interprète ces expressions et les remplace par le SQL correspondant, produisant une requête finale qui est transmise à la base.

#### 4.1.5 Les tests de qualité

dbt intègre un mécanisme de **tests** permettant de vérifier automatiquement des propriétés attendues des données. On distingue deux formes :

- les **tests génériques**, déclarés en configuration : absence de valeurs nulles (`not_null`), unicité d'une colonne (`unique`), ou appartenance à un ensemble de valeurs autorisées (`accepted_values`) ;
- les **tests personnalisés**, exprimés sous forme de requêtes `SELECT`. Par convention, un test réussit si sa requête ne retourne aucune ligne, et échoue si elle en retourne. Une requête de test est donc conçue pour sélectionner les enregistrements *en violation* d'une règle : leur présence fait échouer le test et signale l'anomalie.

Ce mécanisme fait de dbt un véritable moteur de contrôle qualité, dans lequel les règles métier sont exprimées, versionnées et vérifiées automatiquement.

## 4.2 Utilisation dans le projet

dbt constitue le cœur du cadre de gestion de la qualité des données mis en place dans ce projet. Il organise les traitements SQL en couches successives, chacune concrétisée dans un schéma dédié, et applique les règles de qualité sous forme de tests. Sa mission n'est pas de corriger les données destinées au reporting, mais d'en **évaluer et documenter la qualité**, afin de fournir aux responsables une base fiable pour leurs décisions.

### 4.2.1 La couche de préparation (staging)

Sept modèles de préparation, concrétisés en **vues**, lisent les tables brutes via la fonction `source()`. Chaque vue assure deux fonctions : la mise en forme (conversion des types, normalisation des chaînes, harmonisation des adresses électroniques) et le calcul de **drapeaux d'anomalie** propres à chaque ligne.

Ainsi, la vue des clients calcule pour chaque enregistrement un indicateur signalant l'absence de tout moyen de contact valide. La concrétisation en vue est ici justifiée par la légèreté de ces traitements et l'absence de besoin de stockage.

#### 4.2.2 La couche de détection

Quatre modèles, concrétisés en **tables**, s'appuient sur les vues de préparation via la fonction `ref()`. Ils réalisent les **jointures entre tables** nécessaires à la détection des incohérences — par exemple, rapprocher les transactions des comptes pour repérer celles qui référencent un compte absent — et attribuent à chaque enregistrement un **score de qualité**. Cette couche conserve l'ensemble des enregistrements, valides comme anormaux, chacun étant simplement qualifié.

#### 4.2.3 Les règles de qualité

Huit tests personnalisés formalisent les règles métier du projet. Chaque test sélectionne les enregistrements qui violent une règle donnée. À l'exécution, ces tests « échouent » de façon attendue : chaque échec correspond à des anomalies correctement détectées, et le nombre d'enregistrements signalés coïncide exactement avec le nombre d'anomalies présentes dans les données. Ces tests constituent ainsi la démonstration du bon fonctionnement du dispositif de détection.

#### 4.2.4 La construction de l'entrepôt décisionnel

Enfin, dbt construit les deux espaces de l'entrepôt décisionnel, concrétisés en tables dans deux schémas distincts. Le **data mart métier** ne conserve que les enregistrements valides ou acceptés, à l'exclusion des rejets ; le **data mart qualité** rassemble l'intégralité des enregistrements, enrichis de leur qualification et de la décision finale, et sert de source au reporting et au modèle prédictif. Grâce à la résolution automatique des dépendances, l'ensemble de ces couches est construit dans le bon ordre à partir d'une seule commande, ce qui prépare directement l'automatisation ultérieure du pipeline.

## 5 FastAPI

### 5.1 Présentation de la technologie

FastAPI est un cadre de développement (*framework*) Python destiné à la construction d'**interfaces de programmation applicative** (API) web. Une API web est un ensemble de points d'accès, exposés sur le réseau, qu'un client (un navigateur, une application) peut interroger pour consulter ou manipuler des données. FastAPI permet de définir ces points d'accès de manière concise, tout en offrant de très bonnes performances.

### 5.1.1 Le style REST

FastAPI s'inscrit dans le style d'architecture **REST** (*Representational State Transfer*), qui structure une API autour de **ressources** identifiées par des adresses (URL) et manipulées au moyen des verbes du protocole HTTP. Les principaux verbes sont **GET** (lire une ressource), **POST** (créer une ressource), **PUT** (mettre à jour) et **DELETE** (supprimer). Cette organisation rend l'API prévisible et facile à comprendre : l'adresse indique *quelle* ressource est concernée, et le verbe *quelle* action est demandée.

Les échanges entre le client et l'API se font le plus souvent au format **JSON**, un format texte léger et universel, aussi bien compris par le serveur que par le navigateur.

### 5.1.2 Les fonctions de traitement des requêtes

Dans FastAPI, chaque point d'accès est associé à une **fonction Python**. À chaque fois qu'une requête parvient à une adresse donnée, la fonction correspondante est exécutée, et sa valeur de retour est automatiquement convertie en réponse JSON. Le développeur écrit donc une simple fonction Python, sans avoir à gérer manuellement les détails du protocole HTTP.

### 5.1.3 La validation automatique des données

L'une des forces de FastAPI réside dans la **validation automatique** des données reçues. En s'appuyant sur les annotations de type de Python et sur des modèles de données déclaratifs, FastAPI vérifie que les données envoyées par le client sont conformes à ce qui est attendu — présence des champs requis, respect des types, appartenance à des ensembles de valeurs autorisées. Si une donnée est invalide, FastAPI renvoie automatiquement une erreur explicite, sans que le développeur ait à écrire ces vérifications. Cela renforce la robustesse de l'API et réduit les risques d'erreur.

### 5.1.4 La documentation interactive

FastAPI génère automatiquement une **documentation interactive** de l'API, à partir de la définition des points d'accès et des modèles de données. Cette documentation, accessible depuis un navigateur, permet de visualiser tous les points d'accès disponibles et de les tester directement, ce qui facilite le développement et la vérification.

### 5.1.5 Le service de fichiers statiques

Au-delà des points d'accès de données, FastAPI peut également **servir des fichiers statiques** — pages HTML, feuilles de style, scripts JavaScript, images. Cela permet à une même application de fournir à la fois les données (via l'API) et l'interface qui les affiche, sans recourir à un serveur web distinct.

## 5.2 Utilisation dans le projet

FastAPI assure le **backend** de l'interface d'administration de la plateforme. Il joue le rôle d'intermédiaire entre la base de données et l'interface consultée par le responsable : il expose, sous forme de points d'accès, les informations de qualité issues de la base, et reçoit les décisions prises par le responsable.

Les points d'accès sont organisés en trois ensembles cohérents :

- l'ensemble **tableau de bord**, qui expose les indicateurs de synthèse : volumes de données, répartition des anomalies par dimension et par règle ;
- l'ensemble **anomalies**, qui expose la liste des enregistrements en attente de décision pour chaque règle métier, et permet de les inspecter en détail ;
- l'ensemble **décisions**, qui reçoit les décisions du responsable (acceptation ou rejet), enregistre ces décisions dans la base et met en œuvre la propagation des rejets en cascade.

La **validation automatique** des données est mise à profit pour sécuriser les décisions reçues : un modèle de données déclaratif garantit que chaque décision transmise comporte les champs nécessaires et que ceux-ci respectent les valeurs autorisées (par exemple, une décision ne peut être que « acceptée » ou « rejetée », et une sévérité que « faible », « moyenne » ou « élevée »). Toute requête non conforme est rejetée avant même d'atteindre la logique métier.

FastAPI assure par ailleurs le **service des pages de l'interface** : les pages HTML, les feuilles de style et les scripts de l'interface d'administration sont servis par la même application, qui constitue ainsi le point d'entrée unique de la plateforme, accessible depuis un navigateur.

## 6 HTML, CSS et JavaScript

### 6.1 Présentation de la technologie

Les interfaces web reposent sur trois technologies complémentaires, exécutées par le navigateur, chacune assumant un rôle distinct : HTML pour la structure, CSS pour la présentation, et JavaScript pour le comportement.

#### 6.1.1 HTML : la structure

HTML (*HyperText Markup Language*) est le langage qui définit la **structure** et le **contenu** d'une page web. Il organise le contenu au moyen de **balises** qui délimitent des éléments : titres, paragraphes, tableaux, listes, boutons, formulaires. L'ensemble de ces éléments forme une arborescence, appelée **DOM** (*Document Object Model*), qui représente la page sous forme d'un arbre hiérarchique. HTML décrit *ce que contient* la page, sans se préoccuper de son apparence ni de son comportement.

### 6.1.2 CSS : la présentation

CSS (*Cascading Style Sheets*) est le langage qui définit la **présentation** des éléments HTML : couleurs, polices, tailles, marges, positionnement, disposition générale. Le principe du CSS est de **séparer le fond de la forme** : la structure reste décrite en HTML, tandis que toute l'apparence est régie par des règles de style. Une règle CSS associe à un ensemble d'éléments un ensemble de propriétés visuelles. Cette séparation permet de modifier l'apparence d'une interface sans en toucher la structure, et d'assurer une présentation cohérente sur l'ensemble des pages.

### 6.1.3 JavaScript : le comportement

JavaScript est le langage de programmation exécuté par le navigateur, qui apporte l'**interactivité** à la page. Il permet de réagir aux actions de l'utilisateur (clic sur un bouton, saisie dans un champ), de modifier dynamiquement le contenu de la page en manipulant le DOM, et de communiquer avec un serveur sans recharger la page.

Cette dernière capacité repose sur un mécanisme appelé **requête asynchrone** : le script peut interroger une API en arrière-plan, recevoir des données au format JSON, et mettre à jour uniquement la partie concernée de la page, sans interrompre la navigation. C'est ce mécanisme qui rend possibles les interfaces web modernes, fluides et réactives, dans lesquelles les données se rafraîchissent sans rechargement complet.

## 6.2 Utilisation dans le projet

Ces trois technologies constituent l'**interface d'administration** de la plateforme, à travers laquelle le responsable consulte les anomalies et prend ses décisions. L'interface a été conçue de manière volontairement sobre et directe, sans recours à un cadre applicatif lourd, afin de rester légère et facilement maintenable.

La **structure** des pages est décrite en HTML : le tableau de bord, la page de consultation des anomalies et la page d'historique des décisions organisent leurs contenus en tableaux, cartes de synthèse et fenêtres de dialogue.

La **présentation** est assurée par une feuille de style CSS commune, qui définit une identité visuelle cohérente sur l'ensemble des pages et met en forme les différents éléments : indicateurs, tableaux de données, badges de dimension, boutons d'action.

Le **comportement** est pris en charge par JavaScript, qui joue un rôle central dans le fonctionnement de l'interface. À l'ouverture d'une page, le script interroge les points d'accès du backend par des requêtes asynchrones, reçoit les données de qualité au format JSON et construit dynamiquement les tableaux et les indicateurs correspondants. Lorsqu'un responsable prend une décision, le script transmet cette décision au backend, puis rafraîchit la partie concernée de l'affichage. C'est également JavaScript qui met en œuvre, côté interface, la

**prévisualisation de l'impact en cascade** : avant la confirmation d'un rejet, le script interroge le backend pour connaître le nombre d'enregistrements descendants qui seront également rejetés, et présente cet impact au responsable afin qu'il décide en toute connaissance de cause.